

---

# Adding Memory

Momentum & Nesterov

---

## Lesson 3 of the Optimizer Course

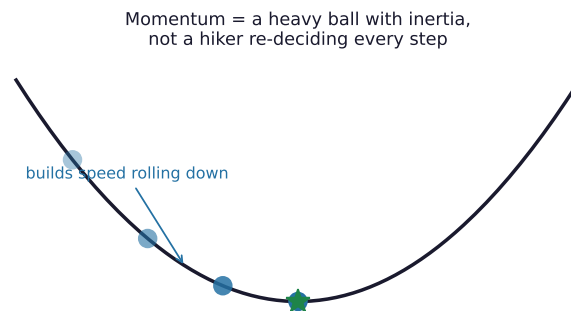
One small change that kills the zig-zag  
and makes descent glide

*Plain gradient descent has amnesia: every step it forgets where it just came from. That's exactly why it bounces wall-to-wall in a lopsided valley. Momentum gives it a memory — it behaves like a heavy ball with inertia. We'll watch that one tweak crush the zig-zag from Lesson 2, with the same numbers, side by side.*

## 1 The problem: a hiker with no memory

At the end of Lesson 2 we watched plain gradient descent bounce back and forth across a stretched valley, barely creeping toward the bottom. Here's *why* it happens: each step, GD looks only at the slope *right now* and forgets everything before. In a narrow valley the “right now” slope mostly points at the near wall, so GD lunges at the wall, overshoots, sees the other wall, lunges back — forever. It has no sense that it's been here before.

A real ball rolling down that valley wouldn't do that. It has **inertia**: it builds up speed in the direction it keeps going, and the side-to-side wobbles cancel out. That's the whole idea of momentum.



### The big idea

Momentum gives the optimizer a **velocity** — a running memory of the recent steps. Instead of stepping along the raw slope, you step along this accumulated velocity. Directions you keep moving build up speed; directions that flip back and forth cancel out. The zig-zag dies; the real progress accelerates.

## 2 The update: just add a velocity

Plain GD was one line. Momentum is two:

$$v \leftarrow \beta v + \nabla f(w) \qquad w \leftarrow w - \eta v$$

**Say it in English:** (1) update your **velocity** — keep **most** of your old velocity and add the new **slope**. (2) step along the **velocity**, not the raw slope.

The new symbol  $\beta$  (“beta,” usually 0.9) is the **momentum coefficient** — how much past velocity you keep.  $\beta = 0$  throws away all memory and you're back to plain GD.  $\beta = 0.9$  means “keep 90% of my momentum and add the new push” — a heavy ball.

## 3 Watch the memory do its job (by hand)

Let's run momentum on the *exact* stretched bowl from Lesson 2,  $f = \frac{1}{2}(10w_1^2 + w_2^2)$ , starting at (1, 1), with  $\eta = 0.12$ ,  $\beta = 0.5$ . The magic is in how the velocity behaves in the two coordinates. Recall the slope is  $\nabla f = (10w_1, w_2)$ .

## Let's actually do the numbers

**The oscillating direction ( $w_1$ ) gets damped.** Track its velocity:

- Step 1:  $\text{slope}_{w_1} = 10$ . Velocity =  $0.5(0) + 10 = \mathbf{10}$ .
- Step 2: now  $w_1$  has overshoot, so  $\text{slope}_{w_1} = -2$  (*flipped sign*). Velocity =  $0.5(10) + (-2) = \mathbf{3}$ . The old +10 memory *cancels* most of the new  $-2$  — the velocity stays positive and small instead of violently reversing.
- Step 3:  $\text{slope}_{w_1} = -5.6$ . Velocity =  $0.5(3) + (-5.6) = \mathbf{-4.1}$ .

The back-and-forth slopes partly cancel inside the velocity, so  $w_1$  stops slamming into the walls.

**The consistent direction ( $w_2$ ) builds speed.** Its slope stays positive the whole way:

$$\text{velocity}_{w_2} : \quad 1.00 \rightarrow 0.5(1.0) + 0.88 = 1.38 \rightarrow 0.5(1.38) + 0.71 = 1.40.$$

The pushes *add up*, so the velocity ( $\approx 1.4$ ) is bigger than any single slope ( $\approx 0.9$ ). Momentum moves *faster* than plain GD ever could in this direction — exactly where GD was crawling.

Put the two runs side by side and the difference is stark:

## Let's actually do the numbers

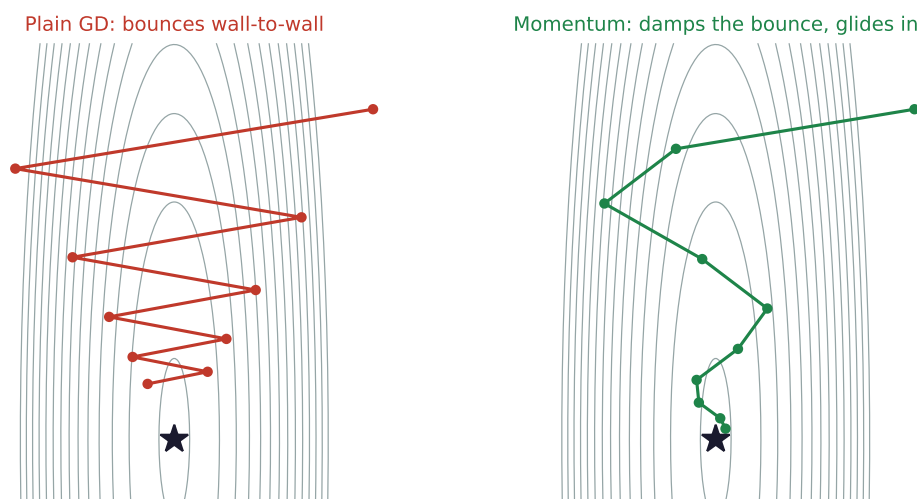
| step | Plain GD ( $\eta = 0.18$ ) | Momentum ( $\eta = 0.12, \beta = 0.5$ ) |
|------|----------------------------|-----------------------------------------|
| 0    | (+1.00, +1.00)             | (+1.00, +1.00)                          |
| 1    | (−0.80, +0.82)             | (−0.20, +0.88)                          |
| 2    | (+0.64, +0.67)             | (−0.56, +0.71)                          |
| 3    | (−0.51, +0.55)             | (−0.07, +0.55)                          |
| 4    | (+0.41, +0.45)             | (+0.26, +0.40)                          |
| 5    | (−0.33, +0.37)             | (+0.11, +0.27)                          |

GD's  $w_1$  flips sign every single step (zig-zag); momentum's settles down. After 12 steps, GD is still a distance 0.115 from the bottom while momentum is at 0.010 — **more than ten times closer**, on the same hill.

## 4 What $\beta$ actually controls

Think of  $\beta$  as the opposite of friction. Two useful rules of thumb:

- **Memory length**  $\approx \frac{1}{1-\beta}$  **steps**. So  $\beta = 0.9$  remembers about the last 10 gradients;  $\beta = 0.99$  remembers about 100. More memory = smoother, but slower to change direction.
- **Effective learning rate**  $\approx \frac{\eta}{1-\beta}$ . With  $\eta = 0.12$  and  $\beta = 0.5$ , that's 0.24 in the consistent direction — bigger than the  $\eta < 0.2$  that plain GD was *forced* to obey. Momentum effectively cranks up the step size in the safe direction without destabilising the steep one.



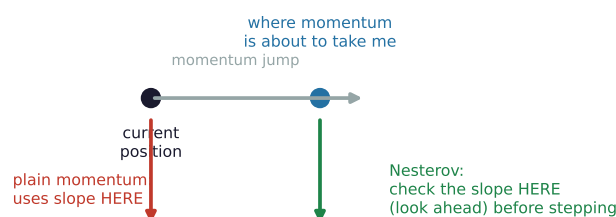
### Watch out (common confusion)

Momentum is not free of risk. Too much ( $\beta$  near 1 with a large  $\eta$ ) and the heavy ball overshoots the bottom and oscillates — it can “roll up the far side.” If a momentum run starts oscillating, lower  $\beta$  or  $\eta$ . The defaults  $\beta = 0.9$  with a tuned  $\eta$  are stable for the vast majority of problems.

## 5 Nesterov: look before you leap

There’s a small, elegant upgrade called **Nesterov Accelerated Gradient (NAG)**. Plain momentum measures the slope where it currently *is*, then jumps. Nesterov first peeks at where the momentum is *about to carry it*, measures the slope *there*, and uses that. It’s the difference between a driver who steers based on the road under the car versus one who looks ahead at the bend coming up.

Nesterov’s one tweak: look before you leap



In symbols, the only change is *where* the gradient is measured:

$$v \leftarrow \beta v + \nabla f\left(\underbrace{w - \eta\beta v}_{\text{the look-ahead point}}\right), \quad w \leftarrow w - \eta v.$$

The payoff is concrete. Running both on our bowl, the first step is identical (no velocity yet to look ahead with), but by step 2 they diverge: plain momentum lands at  $w_1 = -0.20$ , while

Nesterov’s look-ahead sees the overshoot coming and lands gentler at  $w_1 = +0.025$ . That anticipation gives NAG slightly faster, more stable convergence — a free improvement that’s why it’s a standard option wherever momentum is used.

#### When you’re actually training

Momentum costs almost nothing — one extra vector (the velocity) per parameter — and almost always helps, so it’s on by default. “SGD with momentum” ( $\beta = 0.9$ , Nesterov often toggled on) is *still* the workhorse for training vision models from scratch, and frequently reaches better final test accuracy than the fancier adaptive optimizers we’ll meet next. When someone says “just use SGD,” they almost always mean SGD *with momentum*.

## 6 What you can do now, and what’s next

You can now write the momentum update, explain in plain terms why it cancels zig-zag and accelerates real progress, read off what  $\beta$  does (memory length and effective step size), and explain Nesterov’s look-ahead tweak.

**Next — Lesson 4: Per-Parameter Steps.** Momentum helped, but notice we still used *one* learning rate for *every* weight — and the whole problem was that different directions need different step sizes. What if each weight got its *own* step size, automatically shrunk where the slope is wild and grown where it’s calm? That’s the **Adagrad** / **RMSProp** idea, and it’s the foundation of Adam — the optimizer you’ll actually use.